

Lab Exercise 1: Triggering Nested Loop Join

Query Query History

```
1 EXPLAIN (ANALYZE , BUFFERS )
2 SELECT c . first_name , c . last_name , r . rental_date
3 FROM customer c
4 JOIN rental r ON c . customer_id = r . customer_id
5 WHERE c . customer_id = 1 ;
```

Data Output Messages Notifications

Showing rows: 1 to 13 Page No: 1 of 1

	QUERY PLAN	
	text	
1	Nested Loop (cost=0.28..398.93 rows=36 width=21) (actual time=0.582..5.124 rows=32 loops=1)	
2	Buffers: shared read=170 dirtied=18	
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.297..0.321 rows=1 loops=1)	
4	Index Cond: (customer_id = 1)	
5	Buffers: shared read=3	
6	-> Seq Scan on rental r (cost=0.00..390.27 rows=36 width=10) (actual time=0.282..4.785 rows=32 loops=1)	
7	Filter: (customer_id = 1)	
8	Rows Removed by Filter: 16012	
9	Buffers: shared read=167 dirtied=18	
10	Planning:	
11	Buffers: shared hit=187 read=21 dirtied=2	
12	Planning Time: 19.990 ms	
13	Execution Time: 5.572 ms	

- 'customer' table is the Outer, 'rental' table is the Inner
- 'customer' table return 1 row. Hash Join is too expensive because of the startup cost, it has to build a hash table in memory for 'rental' table (more than 16,000 rows).
- the outer 'customer' scan found exactly 1 matching row, it only had to trigger the loop over the 'rental' table once.

Lab Exercise 2: Triggering Hash Join

```
7  EXPLAIN (ANALYZE , BUFFERS )
8  SELECT f.title,l.name as language
9  FROM film f
10 JOIN language l ON f.language_id = l.language_id ;
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.752..2.609 rows=1000 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared read=89 dirtied=1
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.240..1.856 rows=1000 loops=1)
5	Buffers: shared read=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.479..0.480 rows=6 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1 dirtied=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.462..0.463 rows=6 loops=1)
10	Buffers: shared read=1 dirtied=1
11	Planning:
12	Buffers: shared hit=283 read=13 dirtied=1
13	Planning Time: 18.095 ms
14	Execution Time: 3.252 ms

- 'language' table is being hashed, 'film' table is being probed.
- Memory Usage: 9kB
- The execution is the same because hash node only requires 9kB to store 6 rows from table 'language'.

Lab Exercise 3: Triggering Merge Join

```
12 SET enable_hashjoin = OFF ;
13 SET enable_nestloop = OFF ;
14
15 EXPLAIN (ANALYZE , BUFFERS )
16 SELECT f . film_id , f.title,i.inventory_id
17 FROM film f
18 JOIN inventory i ON f . film_id = i . film_id
19 ORDER BY f . film_id ;
```

Data Output Messages Notifications

Showing rows: 1 to 15 Page No: 1 of 1

	QUERY PLAN
1	Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=2.163..2.983 rows=4581 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=86 read=26
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.008..0.199 rows=1000 loops=1)
5	Buffers: shared hit=83 read=1
6	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=1.950..2.078 rows=4581 loops=1)
7	Sort Key: i.film_id
8	Sort Method: quicksort Memory: 336kB
9	Buffers: shared hit=3 read=25
10	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.239..1.506 rows=4581 loops=1)
11	Buffers: shared read=25
12	Planning:
13	Buffers: shared hit=86 read=5 dirtied=1
14	Planning Time: 4.606 ms
15	Execution Time: 3.473 ms

- 'film' table used an Index Scan to get its data pre-sorted for free, the database used a Sequential Scan on the 'inventory' table and then passed it through a Sort node using a quicksort to properly order 'i.film_id' before executing the Merge Join.
- If run RESET ALL; and remove ORDER BY, the execution plan will switch back to a Hash Join. Without forced settings and the strict requirement to output the final results in order, it is much cheaper for the database planner to scan the tables and build a hash table in memory rather than performing the expensive manual sorting operation on the inventory table.

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

```
1 SET enable_indexscan = off;
2
3 EXPLAIN ANALYZE
4 SELECT *
5 FROM rental
6 WHERE rental_date > '2005-05-24';
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

QUERY PLAN
1 Seq Scan on rental (cost=0.00..390.27 rows=17862 width=36) (actual time=0.205..3.287 rows=16044 loop...
2 Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3 Planning Time: 17.653 ms
4 Execution Time: 3.967 ms

```
1 SET enable_indexscan = on;
2
3 EXPLAIN ANALYZE
4 SELECT *
5 FROM rental
6 WHERE rental_date > '2005-05-24';
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

QUERY PLAN
1 Seq Scan on rental (cost=0.00..390.27 rows=17862 width=36) (actual time=0.008..1.110 rows=16044 loop...
2 Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3 Planning Time: 0.162 ms
4 Execution Time: 1.399 ms

Khi tắt `enable_indexscan`, PostgreSQL sử dụng Sequential Scan để đọc bảng rental. Sau khi bật lại Index Scan, PostgreSQL vẫn chọn Sequential Scan vì truy vấn trả về phần lớn số dòng trong bảng, khiến việc đọc tuần tự có chi phí ước tính phù hợp hơn. Hai kế hoạch có cùng estimated cost và số dòng ước tính. Execution Time khác nhau giữa hai lần chạy, có thể do ảnh hưởng của bộ nhớ đệm và điều kiện hệ thống. Kết quả cho thấy Optimizer không chỉ dựa vào việc Index Scan được bật hay tắt mà còn dựa vào chi phí ước tính để lựa chọn kế hoạch

Task 2: Join Strategy Analysis

```

1  EXPLAIN ANALYZE
2  SELECT
3      c.customer_id,
4      c.first_name,
5      c.last_name,
6      r.rental_date,
7      i.inventory_id,
8      i.film_id
9  FROM customer c
10 JOIN rental r
11     ON c.customer_id = r.customer_id
12 JOIN inventory i
13     ON r.inventory_id = i.inventory_id;

```

Data Output
Messages
Notifications

SQL

Showing rows: 1 to 13
Page No: 1 of 1

	QUERY PLAN
	text
1	Hash Join (cost=150.55..590.32 rows=17862 width=31) (actual time=3.739..10.436 rows=16044 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	-> Hash Join (cost=22.48..415.32 rows=17862 width=29) (actual time=1.426..5.427 rows=16044 loops=1)
4	Hash Cond: (r.customer_id = c.customer_id)
5	-> Seq Scan on rental r (cost=0.00..345.62 rows=17862 width=14) (actual time=0.013..0.986 rows=16044 loop=1)
6	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=1.110..1.111 rows=599 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 38kB
8	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.258..0.929 rows=599 loop=1)
9	-> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=2.230..2.231 rows=4581 loops=1)
10	Buckets: 8192 Batches: 1 Memory Usage: 234kB
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.208..1.354 rows=4581 loops=1)
12	Planning Time: 30.787 ms
13	Execution Time: 10.985 ms

```

1  ANALYZE customer;
2  ANALYZE rental;
3  ANALYZE inventory;
4  EXPLAIN ANALYZE
5  SELECT
6      c.customer_id,
7      c.first_name,
8      c.last_name,
9      r.rental_date,
10     i.inventory_id,
11     i.film_id
12 FROM customer c
13 JOIN rental r
14     ON c.customer_id = r.customer_id
15 JOIN inventory i
16     ON r.inventory_id = i.inventory_id;

```

Data Output
Messages
Notifications

Showing rows: 1 to 13
Page No: 1 of 1

QUERY PLAN
1 Hash Join (cost=150.55..562.57 rows=16044 width=31) (actual time=1.394..8.366 rows=16044 loops=1)
2 Hash Cond: (r.inventory_id = i.inventory_id)
3 -> Hash Join (cost=22.48..392.33 rows=16044 width=29) (actual time=0.260..4.366 rows=16044 loops=1)
4 Hash Cond: (r.customer_id = c.customer_id)
5 -> Seq Scan on rental r (cost=0.00..327.44 rows=16044 width=14) (actual time=0.010..0.984 rows=16044 loops=1)
6 -> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.240..0.244 rows=599 loops=1)
7 Buckets: 1024 Batches: 1 Memory Usage: 38kB
8 -> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.006..0.105 rows=599 loops=1)
9 -> Hash (cost=70.81..70.81 rows=4581 width=6) (actual time=1.095..1.096 rows=4581 loops=1)
10 Buckets: 8192 Batches: 1 Memory Usage: 234kB
11 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.006..0.479 rows=4581 loops=1)
12 Planning Time: 0.445 ms
13 Execution Time: 8.768 ms

PostgreSQL đã chọn Hash Join cho cả hai phép JOIN trong truy vấn 3-way JOIN giữa customer, rental và inventory. Sau khi chạy ANALYZE, thuật toán JOIN không thay đổi, nhưng estimated rows đã được cải thiện từ 17,862 xuống 16,044, khớp với số dòng thực tế. Planning Time giảm đáng kể và Execution Time cũng giảm từ 10.985 ms xuống 8.768 ms. Kết quả cho thấy việc cập nhật thống kê giúp Optimizer ước tính dữ liệu chính xác hơn và có thể cải thiện hiệu suất truy vấn.